Lab 7 Report

CSCI 5742: Cybersecurity Programming and Analytics, Spring 2026

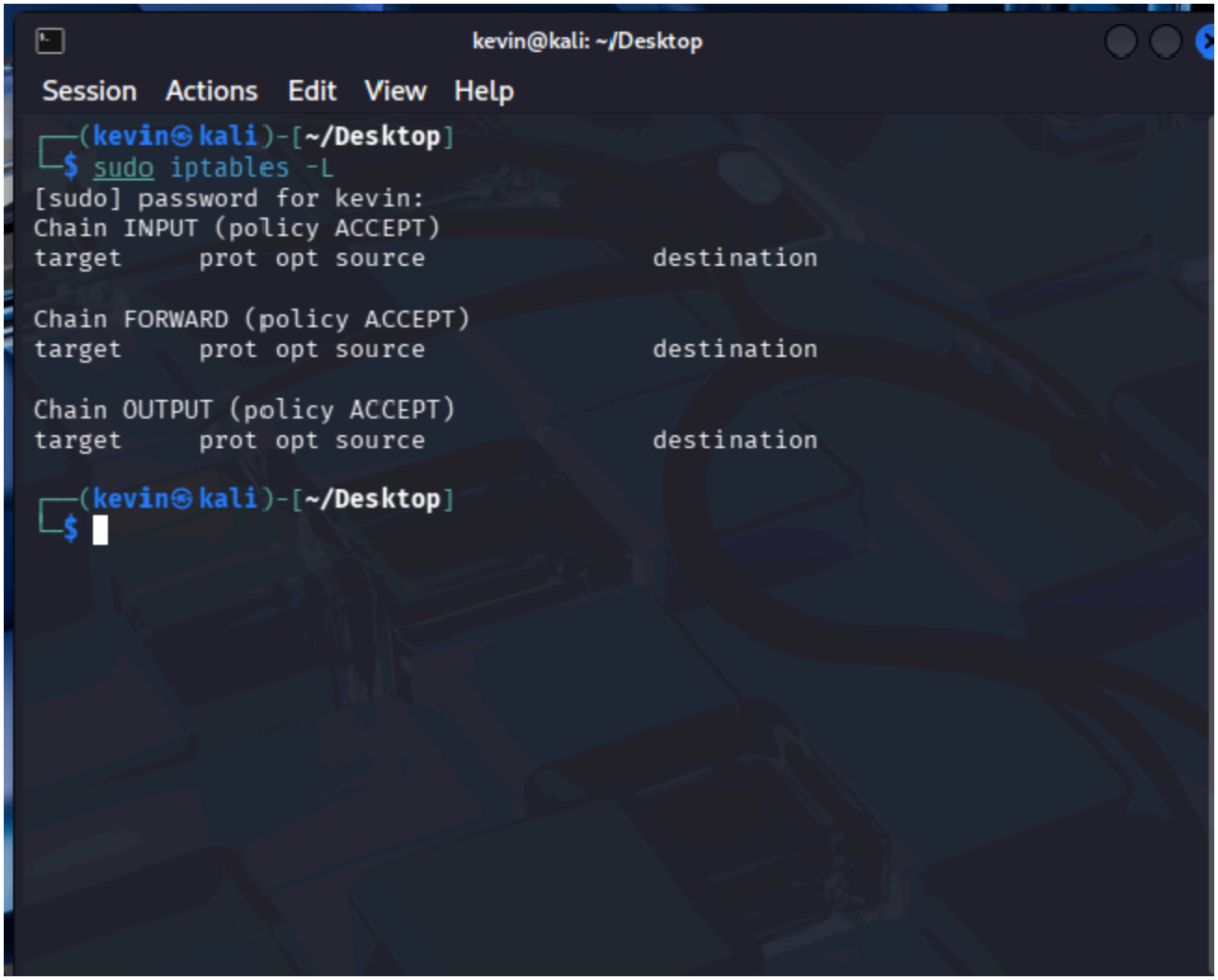
Name & Student ID: Kevin Jacob, 109750578

Task 1: IPTables Basics (10 pts)

Step 1: Understanding IPTables Chains

Screenshot:

(Insert screenshot of `sudo iptables -L` output)



Answers to Questions:

1. In the output, IPTables has three built-in chains. Describe the purpose of each chain. *(Provide your answer here)*

Input handles incoming network packets, output processes the outgoing network packets, and forward handles the packets that are simply passing through tem achine.

2. Explain the difference between **DROP** and **REJECT**. *(Provide your answer here)*

DROP silently discards the packet, whereas REJECT discards the packet but sends an error message back to the sender.

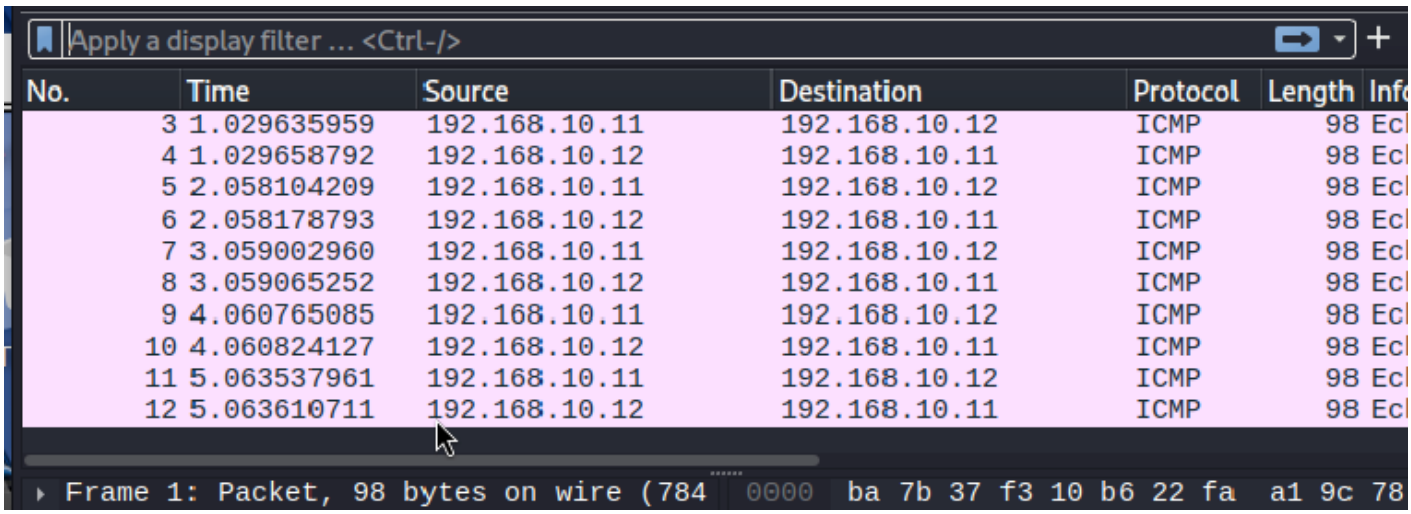
3. Explain when each policy is typically used in a firewall configuration. *(Provide your answer here)*

DROP is the standard for external facing firewalls, REJECT is typically used on trusted internal networks.

Step 2: Testing Network Traffic Before Applying Rules

Screenshot:

(Insert screenshot of Wireshark capture showing ICMP request/reply traffic before applying firewall rules)



No.	Time	Source	Destination	Protocol	Length	Info
3	1.029635959	192.168.10.11	192.168.10.12	ICMP	98	Ec
4	1.029658792	192.168.10.12	192.168.10.11	ICMP	98	Ec
5	2.058104209	192.168.10.11	192.168.10.12	ICMP	98	Ec
6	2.058178793	192.168.10.12	192.168.10.11	ICMP	98	Ec
7	3.059002960	192.168.10.11	192.168.10.12	ICMP	98	Ec
8	3.059065252	192.168.10.12	192.168.10.11	ICMP	98	Ec
9	4.060765085	192.168.10.11	192.168.10.12	ICMP	98	Ec
10	4.060824127	192.168.10.12	192.168.10.11	ICMP	98	Ec
11	5.063537961	192.168.10.11	192.168.10.12	ICMP	98	Ec
12	5.063610711	192.168.10.12	192.168.10.11	ICMP	98	Ec

Answer to Question:

4. Analyze the captured traffic for a pair of PING requests and replies. Explain the ICMP **type** and **code** fields. *(Provide your answer here)*

The ICMP packet from the attack VM has a type 8 and a code 0, indicating that it is a request packet. The packet from the Defense VM has a type 0 and a code 0, indicating that it is a reply packet.

Step 3: Blocking Incoming Traffic on Defense VM

Screenshots:

(Insert screenshot of `iptables` chains after applying the REJECT rule)

```
┌──(kevin㉿kali)-[~/Desktop]
└─$ sudo iptables -A INPUT -s 192.168.10.11 -j REJECT
[sudo] password for kevin:
```

(Insert screenshot of Wireshark capture after applying the REJECT rule)

No.	Time	Source	Destination	Protocol	Length	Info
5	2.010423917	192.168.10.11	192.168.10.12	ICMP	98	Ec
6	2.010485625	192.168.10.12	192.168.10.11	ICMP	126	De
7	3.038046709	192.168.10.11	192.168.10.12	ICMP	98	Ec
8	3.038145043	192.168.10.12	192.168.10.11	ICMP	126	De
9	4.038865210	192.168.10.11	192.168.10.12	ICMP	98	Ec
10	4.038935376	192.168.10.12	192.168.10.11	ICMP	126	De
11	5.044754502	192.168.10.11	192.168.10.12	ICMP	98	Ec
12	5.044830585	192.168.10.12	192.168.10.11	ICMP	126	De
13	6.047617544	192.168.10.11	192.168.10.12	ICMP	98	Ec
14	6.047682836	192.168.10.12	192.168.10.11	ICMP	126	De

Answers to Questions:

5. Do you observe any successful pings? *(Provide your answer here)*

No there are no successful pings after I applied the rule.

6. What error message appears on the Attack VM, and why does it display **"Destination Port Unreachable"**? *(Provide your answer here)*

From 192.168.10.12 icmp_seq=85 Destination Port Unreachable Since I am using REJECT, the sender is notified that the packet was blocked.

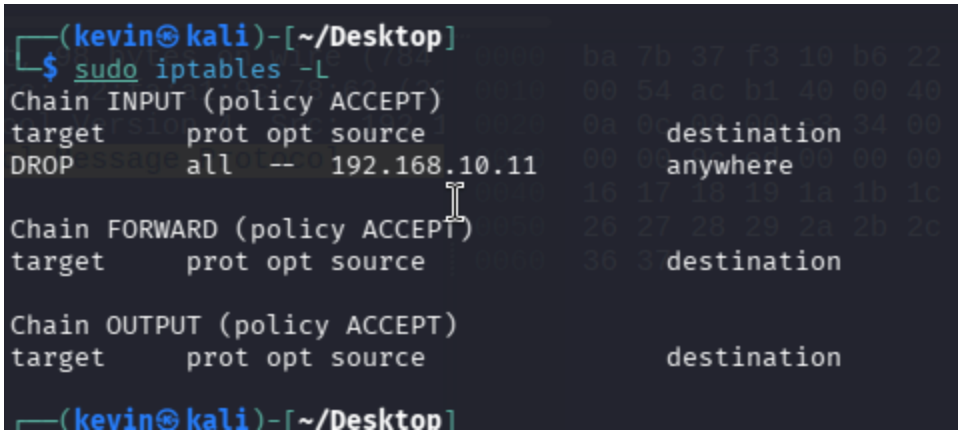
7. Analyze the captured traffic in Wireshark for a PING request and its corresponding reply. Describe any changes in packet behavior. Explain the changes observed in Defense VM's behavior toward PING requests. *(Provide your answer here)*

The Defense VM now replies with a Destination unreachable packet. The firewall is shutting down the ping request at the network layer.

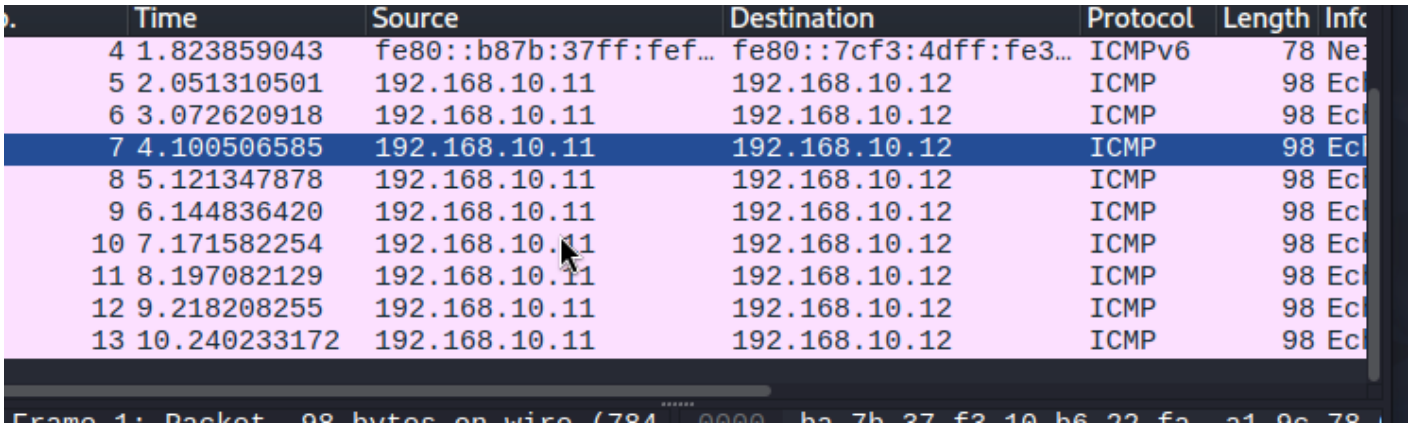
Step 4: Testing DROP vs. REJECT

Screenshots:

(Insert screenshot of `iptables` chains after applying the DROP rule)



(Insert screenshot of Wireshark capture showing behavior with DROP)



	Time	Source	Destination	Protocol	Length	Info
4	1.823859043	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	ICMPv6	78	Ne
5	2.051310501	192.168.10.11	192.168.10.12	ICMP	98	Ec
6	3.072620918	192.168.10.11	192.168.10.12	ICMP	98	Ec
7	4.100506585	192.168.10.11	192.168.10.12	ICMP	98	Ec
8	5.121347878	192.168.10.11	192.168.10.12	ICMP	98	Ec
9	6.144836420	192.168.10.11	192.168.10.12	ICMP	98	Ec
10	7.171582254	192.168.10.11	192.168.10.12	ICMP	98	Ec
11	8.197082129	192.168.10.11	192.168.10.12	ICMP	98	Ec
12	9.218208255	192.168.10.11	192.168.10.12	ICMP	98	Ec
13	10.240233172	192.168.10.11	192.168.10.12	ICMP	98	Ec

Answers to Questions:

8. How does the behavior differ between **DROP** and **REJECT**? *(Provide your answer here)*

When I had a drop rule, the attack VM did not output anything while pinging the unreachable port. However, when I was using REJECT, I was getting constant error messages.

9. Why might a firewall administrator choose one over the other? *(Provide your answer here)*

DROP is almost always used for external traffic, whereas REJECT is used generally for internal trusted networks.

Task 2: Defending Against ICMP & Web Reconnaissance (10 pts)

Step 1: Configuring ICMP Filtering

Screenshots:

(Insert screenshot of `iptables` chains for ICMP filtering) alt text *(Insert screenshot of Wireshark capture for Scenario 1: Attack VM pings Defense VM)*

31984	17787.520663…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0002, seq=9/23
31985	17787.520717…	192.168.10.12	192.168.10.11	ICMP	98	Echo (ping) reply id=0x0002, seq=9/23
31986	17788.548080…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0002, seq=10/2
31987	17788.548179…	192.168.10.12	192.168.10.11	ICMP	98	Echo (ping) reply id=0x0002, seq=10/2
31988	17789.550626…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0002, seq=11/2
31989	17789.550685…	192.168.10.12	192.168.10.11	ICMP	98	Echo (ping) reply id=0x0002, seq=11/2
31990	17790.191860…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	86	Neighbor Solicitation for fe80::b87b:37f
31991	17790.191937…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	ICMPv6	78	Neighbor Advertisement fe80::b87b:37ff:f
31992	17790.555635…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0002, seq=12/3
31993	17790.555694…	192.168.10.12	192.168.10.11	ICMP	98	Echo (ping) reply id=0x0002, seq=12/3
31994	17790.601687…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	ICMPv6	86	Neighbor Solicitation for fe80::7cf3:4df
31995	17790.602134…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	78	Neighbor Advertisement fe80::7cf3:4dff:f
31996	17791.585525…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0002, seq=13/3
31997	17791.585615…	192.168.10.12	192.168.10.11	ICMP	98	Echo (ping) reply id=0x0002, seq=13/3
31998	17792.588124…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0002, seq=14/3
31999	17792.588171…	192.168.10.12	192.168.10.11	ICMP	98	Echo (ping) reply id=0x0002, seq=14/3

(Insert screenshot of Wireshark capture for Scenario 2: Defense VM pings Attack VM)

```
┌──(kevin㉿kali)-[~/Desktop]
└─$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     icmp --  anywhere             anywhere             icmp echo-reply
DROP       icmp --  anywhere             anywhere

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     icmp --  anywhere             anywhere             icmp echo-request
DROP       icmp --  anywhere             anywhere
```

Answers to Questions:

10. Document your observations, focusing on ICMP traffic behavior in each scenario. *(Provide your answer here)*

In scenario 1, the ping is successful and I see the incoming traffic on wireshark. however, in scenario 2 the ping fails and wireshark does not capture anything.

11. How does filtering impact ICMP communication? *(Provide your answer here)*

Filtering enforces strict directionality on network communication. The defense VM is only allowed to act as a responder but it cannot initiate any outgoing queuries.

12. Explain the importance of rule order and why more specific rules (e.g. accept icmp-request) should be defined before broader ones (e.g., drop all icmp). *(Provide your answer here)*

Iptables process rules from top to bottom, so as soon as a packet matches a rule the iptables executes that exaction. So for example, if you were to put a DROP rule for all icmp packets then every single icmp packet would be instantly discarded.

Step 2: Reversing the Policy

Screenshots:

(Insert screenshot of updated `iptables` chains)

32144	17845.927654…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0xc40a A 2.debian.pool.ntp.org
32145	17845.927671…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0xca08 AAAA 2.debian.pool.ntp.org
32146	17845.927923…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	149	Destination Unreachable (Port unreachable)
32147	17845.927981…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0xc40a A 2.debian.pool.ntp.org
32148	17845.927995…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0xca08 AAAA 2.debian.pool.ntp.org
32149	17845.928234…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	149	Destination Unreachable (Port unreachable)
32150	17845.928392…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x8c12 A 3.debian.pool.ntp.org
32151	17845.928409…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x5d12 AAAA 3.debian.pool.ntp.org
32152	17845.928668…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	149	Destination Unreachable (Port unreachable)
32153	17845.928690…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x8c12 A 3.debian.pool.ntp.org
32154	17845.928704…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x5d12 AAAA 3.debian.pool.ntp.org
32155	17845.928952…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	149	Destination Unreachable (Port unreachable)
32156	17850.833607…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	86	Neighbor Solicitation for fe80::b87b:37ff:fef3:10b6 from 7e:
32157	17850.833628…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	ICMPv6	78	Neighbor Advertisement fe80::b87b:37ff:fef3:10b6 (sol)
32158	17851.017653…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	ICMPv6	86	Neighbor Solicitation for fe80::7cf3:4dff:fe3e:6d64 from ba:
32159	17851.017979…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	78	Neighbor Advertisement fe80::7cf3:4dff:fe3e:6d64 (rtr, sol)

(Insert screenshot of Wireshark capture for Scenario 1: Defense VM pings Attack VM)

35200	19259.718647…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0003, seq=1/256, ttl=64
35201	19260.747792…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0003, seq=2/512, ttl=64
35202	19261.770949…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0003, seq=3/768, ttl=64
35203	19262.794956…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0003, seq=4/1024, ttl=64
35204	19263.819317…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0003, seq=5/1280, ttl=64
35205	19264.744807…	22:fa:a1:9c:78:62	ba:7b:37:f3:10:b6	ARP	42	Who has 192.168.10.12? Tell 192.168.10.11
35206	19264.744818…	ba:7b:37:f3:10:b6	22:fa:a1:9c:78:62	ARP	42	192.168.10.12 is at ba:7b:37:f3:10:b6
35207	19264.842050…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0003, seq=6/1536, ttl=64
35208	19265.862912…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0003, seq=7/1792, ttl=64
35209	19266.888913…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0003, seq=8/2048, ttl=64
35210	19267.911789…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0003, seq=9/2304, ttl=64
35211	19268.937787…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0003, seq=10/2560, ttl=64

(Insert screenshot of Wireshark capture for Scenario 2: Attack VM pings Defense VM)

35373	19308.812908…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x4d83 A 2.debian.pool.ntp.org
35374	19308.812916…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0xb083 AAAA 2.debian.pool.ntp.org
35375	19308.813072…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	149	Destination Unreachable (Port unreachable)
35376	19308.813127…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x4d83 A 2.debian.pool.ntp.org
35377	19308.813135…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0xb083 AAAA 2.debian.pool.ntp.org
35378	19308.813278…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	149	Destination Unreachable (Port unreachable)
35379	19308.813445…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x2bbe A 3.debian.pool.ntp.org
35380	19308.813453…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x82be AAAA 3.debian.pool.ntp.org
35381	19308.813577…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	149	Destination Unreachable (Port unreachable)
35382	19308.813600…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x2bbe A 3.debian.pool.ntp.org
35383	19308.813608…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x82be AAAA 3.debian.pool.ntp.org
35384	19308.813808…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	149	Destination Unreachable (Port unreachable)
35385	19309.435683…	192.168.10.12	192.168.10.11	ICMP	98	Echo (ping) request id=0x0002, seq=16/4096, ttl=64 (reply
35386	19309.436428…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) reply id=0x0002, seq=16/4096, ttl=64 (reques
35387	19310.450980…	192.168.10.12	192.168.10.11	ICMP	98	Echo (ping) request id=0x0002, seq=17/4352, ttl=64 (reply
35388	19310.452698…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) reply id=0x0002, seq=17/4352, ttl=64 (reques

Answers to Questions:

13. What differences do you observe compared to the previous test? How does the Defense VM's new policy impact ICMP communication? *(Provide your answer here)*

In scenario 1 the defense VM succesfully pings the attack VM. Scenario 2 now fails because the attack VM can no longer reach the defense VM successfully. This policy establishes an outgoing only connection.

14. Explain the practical significance of this policy in real-world scenarios. *(Provide your answer here)*

This is a common security baseline for servers and workstations. You are effectively reducing the attack surface by blocking incoming pings.

Step 3: Turning Defense VM into a Web Server

Screenshots:

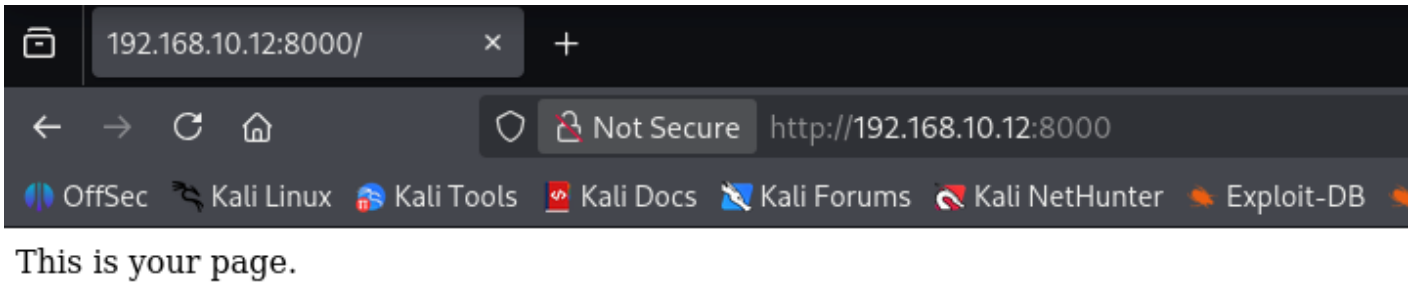
(Insert screenshot showing the web server on port 8000 running)

```
┌──(kevin㉿kali)-[~/Documents/web]
└─$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.10.11 - - [01/Apr/2026 21:43:16] "GET / HTTP/1.1" 200 -
192.168.10.11 - - [01/Apr/2026 21:43:16] code 404, message File not found
192.168.10.11 - - [01/Apr/2026 21:43:16] "GET /favicon.ico HTTP/1.1" 404 -
```

(Insert screenshot showing the second web server on port 8001 running)

```
┌──(kevin㉿kali)-[~/Documents/web]
└─$ python3 -m http.server 8001
Serving HTTP on 0.0.0.0 port 8001 (http://0.0.0.0:8001/) ...
```

(Insert screenshot of browser access from Attack VM to `http://<Defense-VM-IP>:8000`)



(Insert screenshot of `nmap <Defense-VM-IP>` showing both ports 8000 and 8001 open)

```
┌──(kevin㉿kali)-[~/Desktop]
└─$ nmap 192.168.10.12
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-01 21:44 -0600
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 192.168.10.12
Host is up (0.00033s latency).
Not shown: 998 closed tcp ports (reset)
PORT     STATE SERVICE
8000/tcp open  http-alt
8001/tcp open  vcom-tunnel
MAC Address: BA:7B:37:F3:10:B6 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.20 seconds
```

Step 4: Securing HTTP Traffic with IPTables

Screenshots:

(Insert screenshot of running web servers)

```
┌──(kevin㉿kali)-[~/Documents/web]
└─$ python3 -m http.server 8001
Serving HTTP on 0.0.0.0 port 8001 (http://0.0.0.0:8001/) ...
```

(Insert screenshot of `iptables` chains allowing only port 8000)

```
┌──(kevin㉿kali)-[~/Documents/web]
└─$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:8000 state NEW,ESTABLISHED
DROP       tcp  --  anywhere             anywhere

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     tcp  --  anywhere             anywhere             tcp spt:8000 state ESTABLISHED
DROP       tcp  --  anywhere             anywhere

┌──(kevin㉿kali)-[~/Documents/web]
```

(Insert screenshot of Wireshark capture for Scenario 1: access to port 8000)

41132	21080.018382…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x5cdc AAAA 0.debian.pool.ntp.org
41133	21080.018620…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	149	Destination Unreachable (Port unreachable)
41134	21080.018648…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0xb7dd A 0.debian.pool.ntp.org
41135	21080.018661…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x5cdc AAAA 0.debian.pool.ntp.org
41136	21080.018846…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	149	Destination Unreachable (Port unreachable)
41137	21080.018999…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x8a46 A 1.debian.pool.ntp.org
41138	21080.019010…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x2d46 AAAA 1.debian.pool.ntp.org
41139	21080.019218…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	149	Destination Unreachable (Port unreachable)
41140	21080.019280…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x8a46 A 1.debian.pool.ntp.org
41141	21080.019293…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x2d46 AAAA 1.debian.pool.ntp.org
41142	21080.019502…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	149	Destination Unreachable (Port unreachable)
41143	21084.932213…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	86	Neighbor Solicitation for fe80::b87b:37ff:fef3:1
41144	21084.932264…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	ICMPv6	78	Neighbor Advertisement fe80::b87b:37ff:fef3:10b6
41145	21085.134455…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	ICMPv6	86	Neighbor Solicitation for fe80::7cf3:4dff:fe3e:6
41146	21085.136032…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	78	Neighbor Advertisement fe80::7cf3:4dff:fe3e:6d64

(Insert screenshot of Wireshark capture for Scenario 2: access to port 8001)

40771	20930.983631…	192.168.10.11	192.168.10.12	TCP	74	[TCP Retransmission] 38278 → 8001 [SYN] Seq=0 Win=64240 Len=
40772	20931.237430…	192.168.10.11	192.168.10.12	TCP	74	[TCP Retransmission] 38294 → 8001 [SYN] Seq=0 Win=64240 Len=
40773	20932.003180…	192.168.10.11	192.168.10.12	TCP	74	[TCP Retransmission] 38278 → 8001 [SYN] Seq=0 Win=64240 Len=
40774	20932.258880…	192.168.10.11	192.168.10.12	TCP	74	[TCP Retransmission] 38294 → 8001 [SYN] Seq=0 Win=64240 Len=
40775	20932.304345…	ba:7b:37:f3:10:b6	22:fa:a1:9c:78:62	ARP	42	Who has 192.168.10.11? Tell 192.168.10.12
40776	20932.304950…	22:fa:a1:9c:78:62	ba:7b:37:f3:10:b6	ARP	42	192.168.10.11 is at 22:fa:a1:9c:78:62
40777	20933.028458…	192.168.10.11	192.168.10.12	TCP	74	[TCP Retransmission] 38278 → 8001 [SYN] Seq=0 Win=64240 Len=

(Insert screenshot of `nmap <Defense-VM-IP>` showing only port 8000 open)

```
┌──(kevin㉿kali)-[~/Desktop]
└─$ nmap 192.168.10.12
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-01 21:54 -0600
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 192.168.10.12
Host is up (0.00072s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT     STATE SERVICE
8000/tcp open  http-alt
MAC Address: BA:7B:37:F3:10:B6 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 12.26 seconds
```

Answers to Questions:

15. Explain how `iptables -m state --state NEW,ESTABLISHED` functions and why it is crucial for filtering TCP traffic. *(Provide your answer here)*

The -m state allows the firewall to track the connection state. New refers to the first packet of hte connection. ESTABLISHED refers to packets that are part of a connection the firewall has already seen in both directions. It is crucial because it allows the firewall to be stateful ensuring that only legitimate responses are allowed through.

16. Why does the input rule for the allowed port (8000) specify `--state NEW,ESTABLISHED`, while the output rule only includes `--state ESTABLISHED`? *(Provide your answer here)*

The input rule needs NEW to allow the attack VM to initiate the connection to the web server. The output rule only needs established because the server should only be responding to requests that have already been initialized.

17. Based on Wireshark observations, what key differences do you notice when accessing port **8000** versus **8001**? *(Provide your answer here)*

You will see a successful handshake on port 8000. On port 8001 however, you will see the Attack VM send a SYN packet but the defense VM will not provide a response.

18. How does the firewall affect the TCP three-way handshake for allowed connections compared to blocked ones? *(Provide your answer here)*

The firewall permits the initial SYN packet for allowed connections. In a blocked system, the firewall intercepts and dsicards the SYN packet immediately.

Task 3: Rate Limiting & Network Reconnaissance Defense (20 pts)

Step 1: Configuring Basic Rate Limiting

Screenshots:

(Insert screenshot of `iptables` chains with the rate-limiting rule)

```
┌──(kevin㉿kali)-[~/Documents/web]
└─$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     all  --  anywhere             anywhere             limit: avg 1/sec burst 5
DROP       all  --  anywhere             anywhere

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

(Insert screenshot of Wireshark traffic capture during `ping -i 0.1 <Defense-VM-IP>`)

49510	31579.213252…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0005, seq=114/29184, ttl=64 (n
49511	31579.317512…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0005, seq=115/29440, ttl=64 (n
49512	31579.425483…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0005, seq=116/29696, ttl=64 (r
49513	31579.425539…	192.168.10.12	192.168.10.11	ICMP	98	Echo (ping) reply id=0x0005, seq=116/29696, ttl=64 (r
49514	31579.533466…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0005, seq=117/29952, ttl=64 (n
49515	31579.642080…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0005, seq=118/30208, ttl=64 (n
49516	31579.748783…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0005, seq=119/30464, ttl=64 (n
49517	31579.852717…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0005, seq=120/30720, ttl=64 (n
49518	31579.956316…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0005, seq=121/30976, ttl=64 (n
49519	31580.060128…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0005, seq=122/31232, ttl=64 (n
49520	31580.161176…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0005, seq=123/31488, ttl=64 (n
49521	31580.268481…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0005, seq=124/31744, ttl=64 (n
49522	31580.372758…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0005, seq=125/32000, ttl=64 (n
49523	31580.475001…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0005, seq=126/32256, ttl=64 (r
49524	31580.475112…	192.168.10.12	192.168.10.11	ICMP	98	Echo (ping) reply id=0x0005, seq=126/32256, ttl=64 (r
49525	31580.577671…	192.168.10.11	192.168.10.12	ICMP	98	Echo (ping) request id=0x0005, seq=127/32512, ttl=64 (n

Answers to Questions:

19. Explain your observations regarding packet loss and how the rate limit affects traffic. *(Provide your answer here)*

The attack VM is sending 10 packets per second, and the defense VM is only letting some of them through.

20. Does the rate limit completely block all ICMP pings? Why or why not? *(Provide your answer here)*

No it does not block all pings, since the rule is ACCEPT, one ping per second will still go through.

21. Calculate the theoretical and empirical rate of PING packet drops based on network traffic analysis. *(Provide your answer here)*

At 10 packets/sec with a 1 packet/sec limit. The drop is 90%.

22. Why is rate-limiting incoming traffic essential for network security and performance? *(Provide your answer here)*

It prevents DDOS atacks by sending that a single attacker cannot overwhelm the server.

23. What potential challenges or drawbacks could arise when implementing rate-limiting in a real-world network environment? How can these be mitigated? *(Provide your answer here)*

It could accidentally block legitimate high traffic bursts.

Step 2: Testing with Nmap - Aggressive Scan (-T5)

Screenshots:

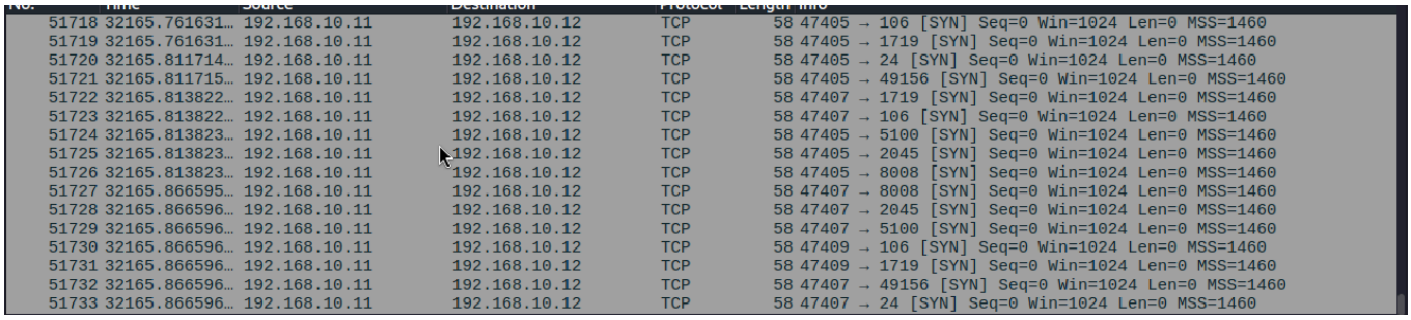
(Insert screenshot of `iptables` chains for this step)

```
┌──(kevin㉿kali)-[~/Documents/web]
└─$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     all  --  anywhere             anywhere             limit: avg 1/sec burst 5
DROP       all  --  anywhere             anywhere

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

(Insert screenshot of Wireshark capture during the aggressive Nmap scan)



No.	Time	Source	Destination	Protocol	Length	Info
51718	32165.761631…	192.168.10.11	192.168.10.12	TCP	58	47405 → 106 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
51719	32165.761631…	192.168.10.11	192.168.10.12	TCP	58	47405 → 1719 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
51720	32165.811714…	192.168.10.11	192.168.10.12	TCP	58	47405 → 24 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
51721	32165.811715…	192.168.10.11	192.168.10.12	TCP	58	47405 → 49156 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
51722	32165.813822…	192.168.10.11	192.168.10.12	TCP	58	47407 → 1719 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
51723	32165.813822…	192.168.10.11	192.168.10.12	TCP	58	47407 → 106 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
51724	32165.813823…	192.168.10.11	192.168.10.12	TCP	58	47405 → 5100 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
51725	32165.813823…	192.168.10.11	192.168.10.12	TCP	58	47405 → 2045 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
51726	32165.813823…	192.168.10.11	192.168.10.12	TCP	58	47405 → 8008 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
51727	32165.866595…	192.168.10.11	192.168.10.12	TCP	58	47407 → 8008 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
51728	32165.866596…	192.168.10.11	192.168.10.12	TCP	58	47407 → 2045 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
51729	32165.866596…	192.168.10.11	192.168.10.12	TCP	58	47407 → 5100 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
51730	32165.866596…	192.168.10.11	192.168.10.12	TCP	58	47409 → 106 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
51731	32165.866596…	192.168.10.11	192.168.10.12	TCP	58	47409 → 1719 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
51732	32165.866596…	192.168.10.11	192.168.10.12	TCP	58	47407 → 49156 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
51733	32165.866596…	192.168.10.11	192.168.10.12	TCP	58	47407 → 24 [SYN] Seq=0 Win=1024 Len=0 MSS=1460

(Optional: insert screenshot of `nmap -T5 <Defense-VM-IP>` output)

Answers to Questions:

24. Does the scan complete successfully? If so, is the result accurate (does it correctly detect port 8000 as open)? *(Provide your answer here)*

The scan failed.

25. Based on observations, how does the scan rate affect the accuracy and detection of open ports? *(Provide your answer here)*

A high scan rate would trigger th firewall's DROP policy leading to massive packet loss. This causes the scanning tool to miss responses.

26. Referencing the Nmap Performance Guide, how do timing templates influence scan behavior and detection efficiency? *(Provide your answer here)*

They adjust variables such as round trip time timeout, parallelism, and delay between probes. Higher templates prioritize speed by assuming a high bandwith, whereas lower templates prioritize evasion by waiting longer between tackets to trigger thresholds.

27. Which Nmap timing profile(s) (`-T0` to `-T4`) could potentially bypass this rate-limiting rule? Justify your answer based on scan behavior and rate thresholds. What is the main drawback of choosing this/these profile(s)? *(Provide your answer here)*

T0 and T1 are the most liekly to bypass the rule since they wait enough between each prode to avoid triggering the DROP policy.

Task 4: Logging and Monitoring Firewall Activity (20 pts)

Step 1: Configuring Firewall Rules to Allow Only HTTP Traffic (Port 8000)

Screenshots:

(Insert screenshot of `iptables` chains)

```
┌──(kevin㉿kali)-[~/Documents/web]
└─$ sudo iptables -F

┌──(kevin㉿kali)-[~/Documents/web]
└─$ sudo iptables -A INPUT -p tcp --dport 8000 -m state --state NEW,ESTABLISHED -j ACCEPT

┌──(kevin㉿kali)-[~/Documents/web]
└─$ sudo iptables -A OUTPUT -p tcp --sport 8000 -m state --state ESTABLISHED -j ACCEPT

┌──(kevin㉿kali)-[~/Documents/web]
└─$ sudo iptables -A INPUT -p tcp -j LOG --log-prefix "IPTables-Dropped: " --log-level 4

┌──(kevin㉿kali)-[~/Documents/web]
└─$ sudo iptables -A INPUT -p tcp -j DROP
```

(Insert screenshot of Wireshark capture for access to port 8000)

No.	Time	Source	Destination	Protocol	Length	Info
57906	33112.321641…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0xf341 AAAA 1.debian.pool.ntp.org
57907	33112.321839…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	149	Destination Unreachable (Port unreachable)
57908	33112.321984…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x6d89 A 2.debian.pool.ntp.org
57909	33112.321992…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x9588 AAAA 2.debian.pool.ntp.org
57910	33112.322203…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	149	Destination Unreachable (Port unreachable)
57911	33112.322222…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x6d89 A 2.debian.pool.ntp.org
57912	33112.322231…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x9588 AAAA 2.debian.pool.ntp.org
57913	33112.322434…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	149	Destination Unreachable (Port unreachable)
57914	33112.322557…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x038c A 3.debian.pool.ntp.org
57915	33112.322565…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0xe58c AAAA 3.debian.pool.ntp.org
57916	33112.322767…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	149	Destination Unreachable (Port unreachable)
57917	33112.322817…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x038c A 3.debian.pool.ntp.org
57918	33112.322827…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0xe58c AAAA 3.debian.pool.ntp.org
57919	33112.323009…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	149	Destination Unreachable (Port unreachable)
57920	33117.053982…	ba:7b:37:f3:10:b6	22:fa:a1:9c:78:62	ARP	42	Who has 192.168.10.11? Tell 192.168.10.12
57921	33117.055026…	22:fa:a1:9c:78:62	ba:7b:37:f3:10:b6	ARP	42	192.168.10.11 is at 22:fa:a1:9c:78:62

(Insert screenshot of Wireshark capture for access to port 8001)

57982	33147.773971…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	ICMPv6	86	Neighbor Solicitation for fe80::7cf3:4dff:fe3e:6d64 from
57983	33147.774944…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	78	Neighbor Advertisement fe80::7cf3:4dff:fe3e:6d64 (rtr, s
57984	33148.307232…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	86	Neighbor Solicitation for fe80::b87b:37ff:fef3:10b6 from
57985	33148.307265…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	ICMPv6	78	Neighbor Advertisement fe80::b87b:37ff:fef3:10b6 (sol)
57986	33161.693494…	192.168.10.11	192.168.10.12	TCP	74	59224 → 8001 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_P
57987	33162.709460…	192.168.10.11	192.168.10.12	TCP	74	[TCP Retransmission] 59224 → 8001 [SYN] Seq=0 Win=64240
57988	33163.730499…	192.168.10.11	192.168.10.12	TCP	74	[TCP Retransmission] 59224 → 8001 [SYN] Seq=0 Win=64240
57989	33164.755634…	192.168.10.11	192.168.10.12	TCP	74	[TCP Retransmission] 59224 → 8001 [SYN] Seq=0 Win=64240
57990	33165.779641…	192.168.10.11	192.168.10.12	TCP	74	[TCP Retransmission] 59224 → 8001 [SYN] Seq=0 Win=64240
57991	33166.805770…	192.168.10.11	192.168.10.12	TCP	74	[TCP Retransmission] 59224 → 8001 [SYN] Seq=0 Win=64240
57992	33166.934357…	22:fa:a1:9c:78:62	ba:7b:37:f3:10:b6	ARP	42	Who has 192.168.10.12? Tell 192.168.10.11
57993	33166.934401…	ba:7b:37:f3:10:b6	22:fa:a1:9c:78:62	ARP	42	192.168.10.12 is at ba:7b:37:f3:10:b6
57994	33168.819870…	192.168.10.11	192.168.10.12	TCP	74	[TCP Retransmission] 59224 → 8001 [SYN] Seq=0 Win=64240

(Insert screenshot of Wireshark capture for `nmap -p 1-100 <Defense-VM-IP>`)

58249	33201.725250…	192.168.10.11	192.168.10.12	TCP	58	43054 → 66 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
58250	33201.827605…	192.168.10.11	192.168.10.12	TCP	58	43056 → 66 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
58251	33201.827606…	192.168.10.11	192.168.10.12	TCP	58	43056 → 45 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
58252	33201.827606…	192.168.10.11	192.168.10.12	TCP	58	43056 → 27 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
58253	33201.827606…	192.168.10.11	192.168.10.12	TCP	58	43056 → 59 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
58254	33201.827606…	192.168.10.11	192.168.10.12	TCP	58	43056 → 99 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
58255	33201.827606…	192.168.10.11	192.168.10.12	TCP	58	43056 → 34 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
58256	33201.827606…	192.168.10.11	192.168.10.12	TCP	58	43056 → 73 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
58257	33201.827606…	192.168.10.11	192.168.10.12	TCP	58	43056 → 95 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
58258	33201.828243…	192.168.10.11	192.168.10.12	TCP	58	43056 → 19 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
58259	33201.828243…	192.168.10.11	192.168.10.12	TCP	58	43056 → 61 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
58260	33203.066326…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x687d A 0.debian.pool.ntp.org
58261	33203.066542…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0xed73 AAAA 0.debian.pool.ntp.org
58262	33203.067413…	fe80::7cf3:4dff:fe3…	fe80::b87b:37ff:fef…	ICMPv6	149	Destination Unreachable (Port unreachable)
58263	33203.067702…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0x687d A 0.debian.pool.ntp.org
58264	33203.067750…	fe80::b87b:37ff:fef…	fe80::7cf3:4dff:fe3…	DNS	101	Standard query 0xed73 AAAA 0.debian.pool.ntp.org

---

Step 2: Viewing and Analyzing Firewall Logs

Screenshot:

(Insert screenshot of `sudo dmesg | grep "IPTables-Dropped:"` or filtered output for port 8001)

```
[22154.404883] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PREC=0x00 TT
L=40 ID=59299 PROTO=TCP SPT=43054 DPT=19 WINDOW=1024 RES=0x00 SYN URGP=0
[22154.405008] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PREC=0x00 TT
L=59 ID=11661 PROTO=TCP SPT=43054 DPT=95 WINDOW=1024 RES=0x00 SYN URGP=0
[22154.405054] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PREC=0x00 TT
L=50 ID=49070 PROTO=TCP SPT=43054 DPT=73 WINDOW=1024 RES=0x00 SYN URGP=0
[22154.405097] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PREC=0x00 TT
L=53 ID=12908 PROTO=TCP SPT=43054 DPT=34 WINDOW=1024 RES=0x00 SYN URGP=0
[22154.405119] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PREC=0x00 TT
L=46 ID=3781 PROTO=TCP SPT=43054 DPT=99 WINDOW=1024 RES=0x00 SYN URGP=0
[22154.405161] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PREC=0x00 TT
L=44 ID=28913 PROTO=TCP SPT=43054 DPT=59 WINDOW=1024 RES=0x00 SYN URGP=0
[22154.405201] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PREC=0x00 TT
L=45 ID=42936 PROTO=TCP SPT=43054 DPT=27 WINDOW=1024 RES=0x00 SYN URGP=0
[22154.405288] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PREC=0x00 TT
L=54 ID=30738 PROTO=TCP SPT=43054 DPT=45 WINDOW=1024 RES=0x00 SYN URGP=0
[22154.405338] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PREC=0x00 TT
L=41 ID=14831 PROTO=TCP SPT=43054 DPT=66 WINDOW=1024 RES=0x00 SYN URGP=0
[22154.507722] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PREC=0x00 TT
L=57 ID=56649 PROTO=TCP SPT=43056 DPT=66 WINDOW=1024 RES=0x00 SYN URGP=0
[22154.507990] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PREC=0x00 TT
L=54 ID=46009 PROTO=TCP SPT=43056 DPT=45 WINDOW=1024 RES=0x00 SYN URGP=0
[22154.508072] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PREC=0x00 TT
L=53 ID=44310 PROTO=TCP SPT=43056 DPT=27 WINDOW=1024 RES=0x00 SYN URGP=0
[22154.508106] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PREC=0x00 TT
L=56 ID=63968 PROTO=TCP SPT=43056 DPT=59 WINDOW=1024 RES=0x00 SYN URGP=0
[22154.508137] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PREC=0x00 TT
L=39 ID=55909 PROTO=TCP SPT=43056 DPT=99 WINDOW=1024 RES=0x00 SYN URGP=0
[22154.508169] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PREC=0x00 TT
L=56 ID=10976 PROTO=TCP SPT=43056 DPT=34 WINDOW=1024 RES=0x00 SYN URGP=0
[22154.508200] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PREC=0x00 TT
L=54 ID=3158 PROTO=TCP SPT=43056 DPT=73 WINDOW=1024 RES=0x00 SYN URGP=0
[22154.508232] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PREC=0x00 TT
L=41 ID=58476 PROTO=TCP SPT=43056 DPT=95 WINDOW=1024 RES=0x00 SYN URGP=0
[22154.508272] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PREC=0x00 TT
L=47 ID=26801 PROTO=TCP SPT=43056 DPT=19 WINDOW=1024 RES=0x00 SYN URGP=0
[22154.508337] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PREC=0x00 TT
L=45 ID=27888 PROTO=TCP SPT=43056 DPT=61 WINDOW=1024 RES=0x00 SYN URGP=0
[22184.124573] IPTables-Dropped: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=60 TOS=0x00 PREC=0x00 TT
L=64 ID=52715 DF PROTO=TCP SPT=59224 DPT=8001 WINDOW=64240 RES=0x00 SYN URGP=0
```

Answers to Questions:

28. What source IP addresses are attempting to connect? *(Provide your answer here)*

The source IP is the attack VM, 192.168.10.11

29. What ports are being targeted? *(Provide your answer here)* The port numbers being targeted seem to happen in a random order, but they mostly stay within the 1-100 range.
30. Modify the log search command to filter packets related to **port 8001**. How many entries are recorded in the logs? *(Provide your answer here)*

There are 11 entries that target port 8001.

31. How does logging contribute to identifying and mitigating suspicious activity? *(Provide your answer here)*

FIrewall logs create an audit trail that reveals information about the nature of the network traffic. Using this information you can determine which services or vulnerabilities they are trying to target. In order to mitigate this, admins can proactively update the firewall rules to permanently drop all traffic from the malicious IP once you have identified a pattern in the logs.

Task 5: Filtering and Logging Network Attacks (40 pts)

Step 1: Configuring Rate-Limiting and Logging for ICMP Floods

Screenshots:

(Insert screenshot of `iptables` chains) *(Insert screenshot of Attack VM normal ping output)*

```
┌──(kevin㉿kali)-[~/Desktop]
└─$ ping -c 5 192.168.10.12
PING 192.168.10.12 (192.168.10.12) 56(84) bytes of data.
64 bytes from 192.168.10.12: icmp_seq=1 ttl=64 time=1.46 ms
64 bytes from 192.168.10.12: icmp_seq=2 ttl=64 time=1.69 ms
64 bytes from 192.168.10.12: icmp_seq=3 ttl=64 time=1.93 ms
64 bytes from 192.168.10.12: icmp_seq=4 ttl=64 time=2.02 ms
64 bytes from 192.168.10.12: icmp_seq=5 ttl=64 time=1.42 ms

--- 192.168.10.12 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4013ms
rtt min/avg/max/mdev = 1.419/1.702/2.018/0.242 ms
```

(Insert screenshot of Attack VM flood ping output)

```
└─$ ping -f -i 0.002 192.168.10.12
PING 192.168.10.12 (192.168.10.12) 56(84) bytes of data.
.....................................................................
.....................................................................
.....................................................................
.....................................................................
.....................................................................
.....................................................................
.....................................................................
```

(Insert screenshot of Wireshark traffic capture during the ICMP flood)

2577	174.963105750	192.168.10.11	192.168.10.12	ICMP	98 Ec
2578	174.975102166	192.168.10.11	192.168.10.12	ICMP	98 Ec
2579	174.987900333	192.168.10.11	192.168.10.12	ICMP	98 Ec
2580	174.999175500	192.168.10.11	192.168.10.12	ICMP	98 Ec
2581	175.009276416	192.168.10.11	192.168.10.12	ICMP	98 Ec
2582	175.019600916	192.168.10.11	192.168.10.12	ICMP	98 Ec
2583	175.030[illegible]90541	192.168.10.11	192.168.10.12	ICMP	98 Ec
2584	175.042099458	192.168.10.11	192.168.10.12	ICMP	98 Ec
2585	175.054426583	192.168.10.11	192.168.10.12	ICMP	98 Ec
2586	175.066085833	192.168.10.11	192.168.10.12	ICMP	98 Ec

(Insert screenshot of `dmesg` logs for `ICMP Flood Detected:`)

```
00 TTL=64 ID=41355 DF PROTO=ICMP TYPE=8 CODE=0 ID=7 SEQ=4561
[  357.835687] ICMP Flood Detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:
a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=84 TOS=0x00 PREC=0x
00 TTL=64 ID=41358 DF PROTO=ICMP TYPE=8 CODE=0 ID=7 SEQ=4562
[  357.847263] ICMP Flood Detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:
a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=84 TOS=0x00 PREC=0x
00 TTL=64 ID=41361 DF PROTO=ICMP TYPE=8 CODE=0 ID=7 SEQ=4563
[  357.858687] ICMP Flood Detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:
a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=84 TOS=0x00 PREC=0x
00 TTL=64 ID=41363 DF PROTO=ICMP TYPE=8 CODE=0 ID=7 SEQ=4564
[  357.868771] ICMP Flood Detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:
a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=84 TOS=0x00 PREC=0x
00 TTL=64 ID=41364 DF PROTO=ICMP TYPE=8 CODE=0 ID=7 SEQ=4565
[  357.879364] ICMP Flood Detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:
a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=84 TOS=0x00 PREC=0x
00 TTL=64 ID=41366 DF PROTO=ICMP TYPE=8 CODE=0 ID=7 SEQ=4566
[  357.889573] ICMP Flood Detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:
a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=84 TOS=0x00 PREC=0x
00 TTL=64 ID=41368 DF PROTO=ICMP TYPE=8 CODE=0 ID=7 SEQ=4567
[  357.900931] ICMP Flood Detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:
a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=84 TOS=0x00 PREC=0x
00 TTL=64 ID=41369 DF PROTO=ICMP TYPE=8 CODE=0 ID=7 SEQ=4568
[  357.913707] ICMP Flood Detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:
a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=84 TOS=0x00 PREC=0x
00 TTL=64 ID=41372 DF PROTO=ICMP TYPE=8 CODE=0 ID=7 SEQ=4569

┌──(kevin㉿kali)-[~/Desktop]
```

Answers to Questions:

32. How many ICMP packets were logged as exceeding the rate limit? *(Provide your answer here)*

Almost all the packets were flagged and dropped. Since my command should technically send 500 packets per second, the firewall only allows 1 packet per second.

33. How did the Defense VM handle the attack? *(Provide your answer here)*

It stopped it and only accepted 1 packet per second even though the attack was designed to output 500 packets per second.

34. How would attackers try to bypass this defense? *(Provide your answer here)*

Since this rule currently only drops icmp traffic, an attacker could switch protocols and send a different packet type to flood the server's resources.

Step 2: Detecting and Logging Regular Nmap Scans

Screenshots:

(Insert screenshot of `iptables` chains for Nmap scan detection)

```
┌──(kevin㉿kali)-[~/Desktop]
└─$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
           tcp  --  anywhere             anywhere             tcp flags:FIN,S
YN,RST,ACK/SYN recent: SET name: nmapscan side: source mask: 255.255.255.255
LOG        tcp  --  anywhere             anywhere             tcp flags:FIN,S
YN,RST,ACK/SYN recent: CHECK seconds: 10 hit_count: 5 name: nmapscan side: so
urce mask: 255.255.255.255 LOG level warn prefix "Nmap scan detected: "
DROP       tcp  --  anywhere             anywhere             tcp flags:FIN,S
YN,RST,ACK/SYN recent: CHECK seconds: 10 hit_count: 5 name: nmapscan side: so
urce mask: 255.255.255.255

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

(Insert screenshot of `nmap -sS -p 1-100 <Defense-VM-IP>` output on Attack VM)

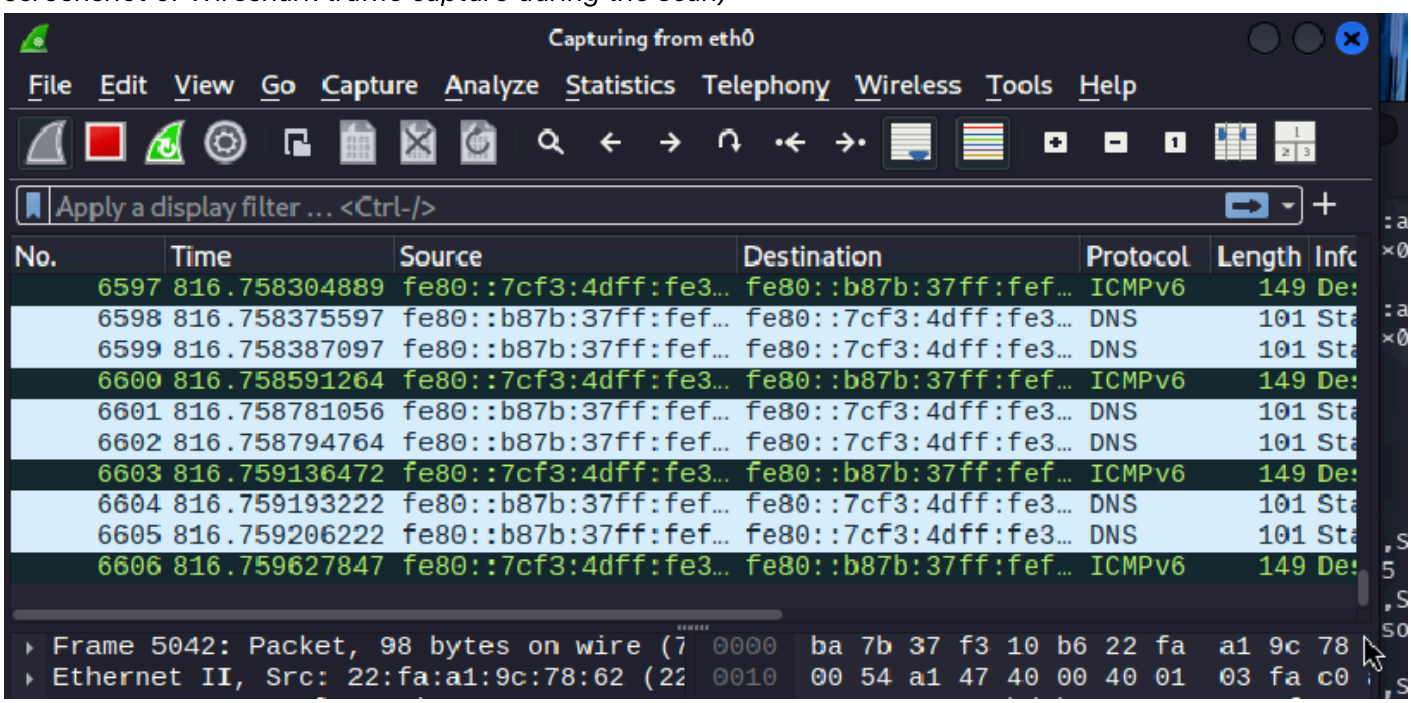
```
┌──(kevin㉿kali)-[~/Desktop]
└─$ nmap -sS -p 1-100 192.168.10.12
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-02 01:36 -0600
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --sys
tem-dns or specify valid servers with --dns-servers
Nmap scan report for 192.168.10.12
Host is up (0.00069s latency).
Not shown: 96 filtered tcp ports (no-response)
PORT   STATE  SERVICE
21/tcp closed ftp
23/tcp closed telnet
25/tcp closed smtp
80/tcp closed http
MAC Address: BA:7B:37:F3:10:B6 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 16.76 seconds
```

(Insert screenshot of `dmesg` logs for `Nmap scan detected:`)

```
0 TTL=43 ID=52695 PROTO=TCP SPT=33903 DPT=95 WINDOW=1024 RES=0×00 SYN URGP=0
[  807.195844] Nmap scan detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a
1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0×00 PREC=0×0
0 TTL=40 ID=14209 PROTO=TCP SPT=33905 DPT=95 WINDOW=1024 RES=0×00 SYN URGP=0
[  807.300239] Nmap scan detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a
1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0×00 PREC=0×0
0 TTL=59 ID=63423 PROTO=TCP SPT=33903 DPT=75 WINDOW=1024 RES=0×00 SYN URGP=0
[  807.404406] Nmap scan detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a
1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0×00 PREC=0×0
0 TTL=52 ID=2668 PROTO=TCP SPT=33930 DPT=23 WINDOW=1024 RES=0×00 SYN URGP=0
[  807.504796] Nmap scan detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a
1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0×00 PREC=0×0
0 TTL=39 ID=4869 PROTO=TCP SPT=33905 DPT=75 WINDOW=1024 RES=0×00 SYN URGP=0
[  807.608863] Nmap scan detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a
1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0×00 PREC=0×0
0 TTL=44 ID=14874 PROTO=TCP SPT=33903 DPT=4 WINDOW=1024 RES=0×00 SYN URGP=0
[  807.713581] Nmap scan detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a
1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0×00 PREC=0×0
0 TTL=46 ID=53639 PROTO=TCP SPT=33905 DPT=4 WINDOW=1024 RES=0×00 SYN URGP=0
[  807.815013] Nmap scan detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a
1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0×00 PREC=0×0
0 TTL=53 ID=38576 PROTO=TCP SPT=33903 DPT=1 WINDOW=1024 RES=0×00 SYN URGP=0
[  807.915132] Nmap scan detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a
1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0×00 PREC=0×0
0 TTL=53 ID=18843 PROTO=TCP SPT=33905 DPT=1 WINDOW=1024 RES=0×00 SYN URGP=0
```

(Insert screenshot of Wireshark traffic capture during the scan)



Answers to Questions:

35. How many scan attempts were logged as exceeding the threshold? *(Provide your answer here)*

Since the firewall allows 5 packets, the other 95 packets were logged and dropped by the firewall.

36. Was the scan able to identify any open ports, or was it blocked? *(Provide your answer here)*

The scan returned 4 specific closed ports and labelled the other 96 ports as "filtered"

37. How does rate-limiting SYN packets help prevent port scanning? *(Provide your answer here)*

Port scanning requires rapidly sending SYN packets to map a network's attack surface. Rate limiting these packets destroys a scanners ability to do that.

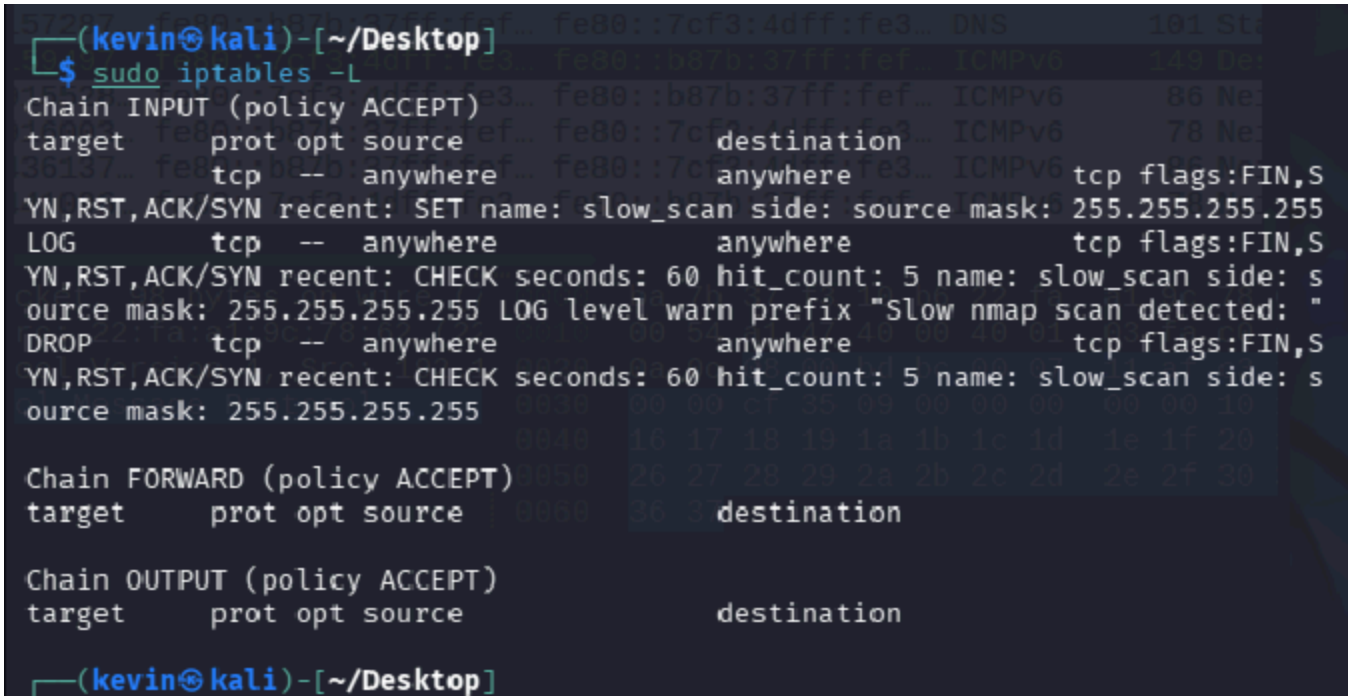
38. What modifications could an attacker use to bypass this detection mechanism? *(Provide your answer here)*

An attack could simply use a T0 or T1 to wait longer than 10 seconds between each packet.

Step 3: Testing Slow and Stealthy Nmap Scans

Screenshots:

(Insert screenshot of `iptables` chains for slow scan detection)



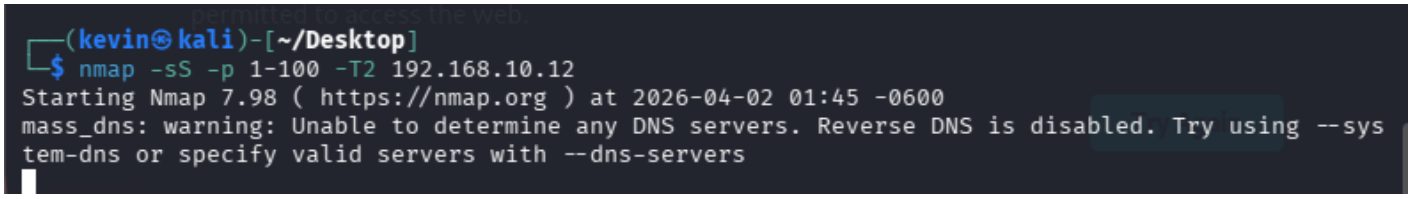
```
┌──(kevin㉿kali)-[~/Desktop]
└─$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
           tcp  --  anywhere             anywhere             tcp flags:FIN,S
YN,RST,ACK/SYN recent: SET name: slow_scan side: source mask: 255.255.255.255
LOG        tcp  --  anywhere             anywhere             tcp flags:FIN,S
YN,RST,ACK/SYN recent: CHECK seconds: 60 hit_count: 5 name: slow_scan side: s
ource mask: 255.255.255.255 LOG level warn prefix "Slow nmap scan detected: "
DROP       tcp  --  anywhere             anywhere             tcp flags:FIN,S
YN,RST,ACK/SYN recent: CHECK seconds: 60 hit_count: 5 name: slow_scan side: s
ource mask: 255.255.255.255

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination

┌──(kevin㉿kali)-[~/Desktop]
```

(Insert screenshot of `nmap -sS -p 1-100 -T2 <Defense-VM-IP>` output on Attack VM)



```
┌──(kevin㉿kali)-[~/Desktop]
└─$ nmap -sS -p 1-100 -T2 192.168.10.12
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-02 01:45 -0600
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --sys
tem-dns or specify valid servers with --dns-servers
```

(Insert screenshot of `dmesg` logs for `Slow Nmap scan detected:`)

```
GP=0
[ 1412.326456] Slow nmap scan detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22
:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PRE
C=0x00 TTL=55 ID=64176 PROTO=TCP SPT=45349 DPT=53 WINDOW=1024 RES=0x00 SYN UR
GP=0
[ 1413.126838] Slow nmap scan detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22
:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PRE
C=0x00 TTL=45 ID=5089 PROTO=TCP SPT=45322 DPT=72 WINDOW=1024 RES=0x00 SYN URG
P=0
[ 1413.932026] Slow nmap scan detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22
:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PRE
C=0x00 TTL=59 ID=2849 PROTO=TCP SPT=45324 DPT=72 WINDOW=1024 RES=0x00 SYN URG
P=0
[ 1414.735036] Slow nmap scan detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22
:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PRE
C=0x00 TTL=59 ID=42323 PROTO=TCP SPT=45322 DPT=73 WINDOW=1024 RES=0x00 SYN UR
GP=0
[ 1415.537975] Slow nmap scan detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22
:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PRE
C=0x00 TTL=50 ID=62108 PROTO=TCP SPT=45324 DPT=73 WINDOW=1024 RES=0x00 SYN UR
GP=0
[ 1416.340177] Slow nmap scan detected: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22
:fa:a1:9c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=44 TOS=0x00 PRE
C=0x00 TTL=41 ID=11538 PROTO=TCP SPT=45322 DPT=82 WINDOW=1024 RES=0x00 SYN UR
GP=0

┌──(kevin㉿kali)-[~/Desktop]
```

Answers to Questions:

39. How does the detection of a slow scan compare to a rapid scan? *(Provide your answer here)*

Rapid scans trigger threshold rules almost instantly because hundreds of packets arrive in a few seconds. Detecting a slow scan requires the firewall to maintain a much longer tracking window.

40. Was the slow scan still able to identify open ports? *(Provide your answer here)*

It might sucessfully ping the first couple of ports it tries, but the rest will return as filtered.

41. What modifications could an attacker use to further evade detection? *(Provide your answer here)*

To bypass the rule I set, an attacker must send fewer than 5 packets per minute.

42. How can defensive rules be improved to account for stealthier attacks? *(Provide your answer here)*

Relying solely on IpTables rate limiting is difficult because you cannot track IP states indefinitely without exhausting server memory. To improve defenses, admins should implement IDS/IPS tools. These can analyze logs over much longer periods.

Step 4: Detecting, Logging, and Filtering SSH Brute-Force Attacks

Screenshots:

(Insert screenshot of `iptables` chains for SSH brute-force detection)

```
┌──(kevin㉿kali)-[~/Desktop]
└─$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
           tcp  --  anywhere             anywhere             tcp flags:FIN,S
YN,RST,ACK/SYN recent: SET name: slow_scan side: source mask: 255.255.255.255
LOG        tcp  --  anywhere             anywhere             tcp flags:FIN,S
YN,RST,ACK/SYN recent: CHECK seconds: 60 hit_count: 5 name: slow_scan side: s
ource mask: 255.255.255.255 LOG level warn prefix "Slow nmap scan detected: "
DROP       tcp  --  anywhere             anywhere             tcp flags:FIN,S
YN,RST,ACK/SYN recent: CHECK seconds: 60 hit_count: 5 name: slow_scan side: s
ource mask: 255.255.255.255
           tcp  --  anywhere             anywhere             tcp dpt:ssh fla
gs:FIN,SYN,RST,ACK/SYN recent: SET name: sshscan side: source mask: 255.255.2
55.255
LOG        tcp  --  anywhere             anywhere             tcp dpt:ssh fla
gs:FIN,SYN,RST,ACK/SYN recent: CHECK seconds: 10 hit_count: 5 name: sshscan s
ide: source mask: 255.255.255.255 LOG level warn prefix "SSH brute-force dete
cted: "
DROP       tcp  --  anywhere             anywhere             tcp dpt:ssh fla
gs:FIN,SYN,RST,ACK/SYN recent: CHECK seconds: 10 hit_count: 5 name: sshscan s
ide: source mask: 255.255.255.255
```

(Insert screenshot of legitimate SSH output on Attack VM)

```
┌──(kevin㉿kali)-[~/Desktop]
└─$ ssh 192.168.10.12
The authenticity of host '192.168.10.12 (192.168.10.12)' can't be established.
ED25519 key fingerprint is: SHA256:JvWC57xNTfF1wRiyGUCTwRUgp7BNGldzkNnU/Q90Plk
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.10.12' (ED25519) to the list of known hosts.
kevin@192.168.10.12's password:
Linux kali 6.16.8+kali-arm64 #1 SMP PREEMPT Kali 6.16.8-1kali1 (2025-09-24) aarch64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
```

(Insert screenshot of `nmap -p 22 --script=ssh-brute <Defense-VM-IP>` output on Attack VM)

```
┌──(kevin㉿kali)-[~]
└─$ nmap -p 22 --script=ssh-brute 192.168.10.12
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-02 01:53 MDT
NSE: [ssh-brute] Trying username/password pair: root:root
NSE: [ssh-brute] Trying username/password pair: admin:admin
```

(Insert screenshot of `dmesg` logs for `SSH brute-force detected:`)

```
┌──(kevin㉿kali)-[~/Desktop]
└─$ sudo dmesg | grep "SSH brute-force detected: "

┌──(kevin㉿kali)-[~/Desktop]
└─$
```

Answers to Questions:

43. How many SSH connection attempts were logged before the attacker was blocked? *(Provide your answer here)*

An IP address is allowed 5 attempts before the threshhold is breached.

44. Did the Defense VM successfully block further attempts after the threshold was exceeded? *(Provide your answer here)*

Yes once the hitcount of 5 was reached within the time frame, the DROP rule was triggered and blocked all SSH attempts after.

45. How might attackers attempt to bypass this defense mechanism? *(Provide your answer here)*

They might build a script to only guess every 15 seconds. Since this is outside the 10 second window the counter would reset.

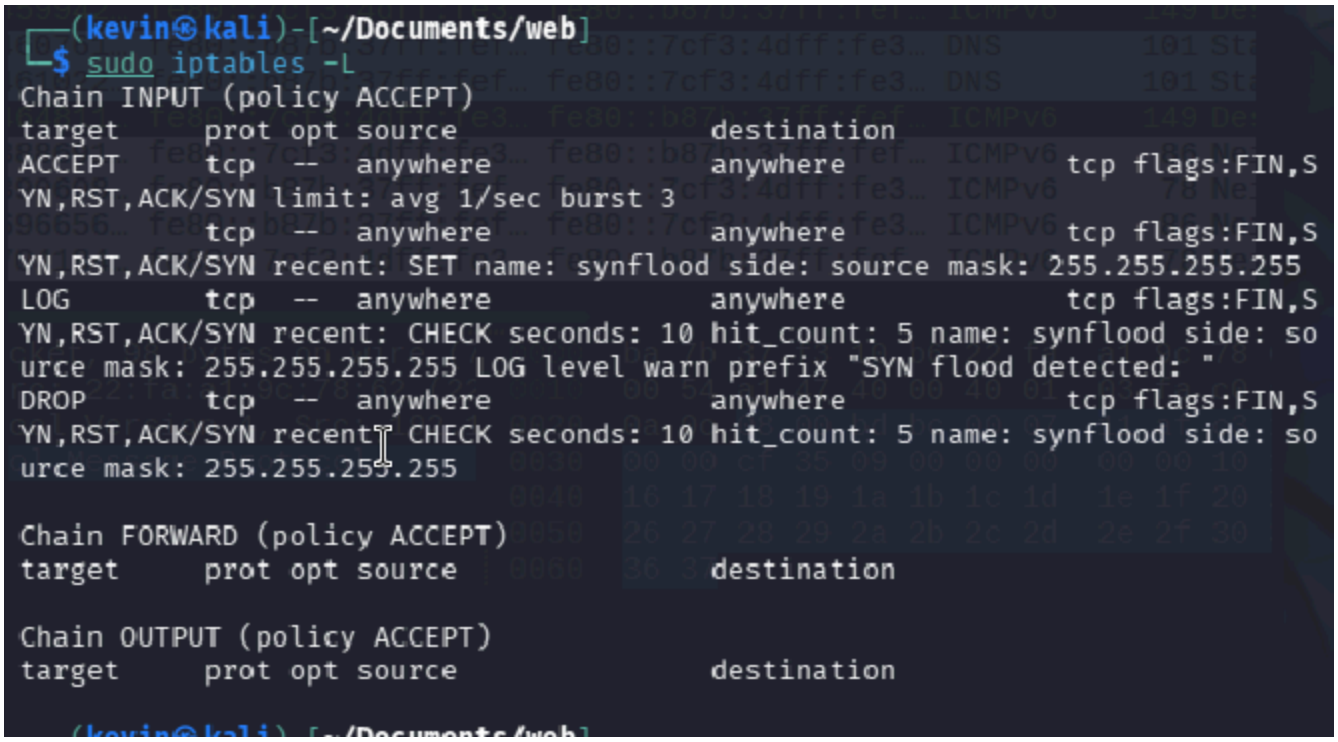
46. What additional measures can enhance SSH security beyond rate-limiting? *(Provide your answer here)*

You can disable password logins entirely and only rqeuire crytographic SSH keys. You can force users to log in with standard privileges and escalata via sudo.

Step 5: Detecting, Logging, and Filtering SYN Flood Attacks

Screenshots:

(Insert screenshot of `iptables` chains for SYN flood detection)



```
┌──(kevin㉿kali)-[~/Documents/web]
└─$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     tcp  --  anywhere             anywhere             tcp flags:FIN,S
YN,RST,ACK/SYN limit: avg 1/sec burst 3
           tcp  --  anywhere             anywhere             tcp flags:FIN,S
YN,RST,ACK/SYN recent: SET name: synflood side: source mask: 255.255.255.255
LOG        tcp  --  anywhere             anywhere             tcp flags:FIN,S
YN,RST,ACK/SYN recent: CHECK seconds: 10 hit_count: 5 name: synflood side: so
urce mask: 255.255.255.255 LOG level warn prefix "SYN flood detected: "
DROP       tcp  --  anywhere             anywhere             tcp flags:FIN,S
YN,RST,ACK/SYN recent: CHECK seconds: 10 hit_count: 5 name: synflood side: so
urce mask: 255.255.255.255

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

(Insert screenshot of legitimate TCP connection output on Attack VM)

```
┌──(kevin㉿kali)-[~]
└─$ curl 192.168.10.12:8000
<HTML><BODY>This is your page.</BODY></HTML>
```

(Insert screenshot of `hping3 --syn --flood <Defense-VM-IP>` output on Attack VM)

```
┌──(kevin㉿kali)-[~]
└─$ hping3 --syn --flood --rand-source 192.168.10.12
[open_sockraw] socket(): Operation not permitted
[main] can't open raw socket
```

(Insert screenshot of `hping3 --syn --flood --rand-source <Defense-VM-IP>` output on Attack VM)

```
┌──(kevin㉿kali)-[~]
└─$ hping3 --syn --flood --rand-source 192.168.10.12
[open_sockraw] socket(): Operation not permitted
[main] can't open raw socket
```

(Insert screenshot of `dmesg` logs for `SYN flood detected:`)

```
┌──(kevin㉿kali)-[~/Documents/web]
└─$ sudo dmesg | grep "SYN flood detected: "
```

Answers to Questions:

47. How many SYN packets were logged before the Defense VM blocked further requests? *(Provide your answer here)*

Based on the rule, only the 5 SYN packets are tracked before the threshold is breached.

48. Did the Defense VM successfully mitigate the SYN flood attack while allowing normal connections? *(Provide your answer here)*

Yes I believe so, the attack VM did not show any success.

49. Why with `--rand-source`, the detection fails? Explain. *(Provide your answer here)*

Since the source is random, every packet is sent with a differnet IP which should bypass the rule since hte firewall never sees 5 packets from the same IP.

50. What alternative techniques could attackers use to evade this SYN flood protection? *(Provide your answer here)*

A distributed DDOS attack, by using multiple machines each machine can send a low rate of SYN Packets which stay below the firewall's radar. But eventually the sheer volume of requests should overwhelm the server.

51. What additional measures can be taken to improve SYN flood mitigation? *(Provide your answer here)*

Using external services can absorb high volume attacks before they reach the local server.

Step 6: Detecting, Logging, and Filtering High-Rate HTTP Attacks

Screenshots:

(Insert screenshot of `iptables` chains for HTTP flood detection)

```
YN,RST,ACK/SYN limit: avg 1/sec burst 3
        tcp  --  anywhere             anywhere             tcp flags:FIN,S
YN,RST,ACK/SYN recent: SET name: synflood side: source mask: 255.255.255.255
LOG     tcp  --  anywhere             anywhere             tcp flags:FIN,S
YN,RST,ACK/SYN recent: CHECK seconds: 10 hit_count: 5 name: synflood side: so
urce mask: 255.255.255.255 LOG level warn prefix "SYN flood detected: "
DROP    tcp  --  anywhere             anywhere             tcp flags:FIN,S
YN,RST,ACK/SYN recent: CHECK seconds: 10 hit_count: 5 name: synflood side: so
urce mask: 255.255.255.255
ACCEPT  tcp  --  anywhere             anywhere             tcp dpt:http li
mit: avg 10/sec burst 20
        tcp  --  anywhere             anywhere             tcp dpt:http re
cent: SET name: httpflood side: source mask: 255.255.255.255
LOG     tcp  --  anywhere             anywhere             tcp dpt:http re
cent: CHECK seconds: 30 hit_count: 15 name: httpflood side: source mask: 255.
255.255.255 LOG level warn prefix "HTTP flood detected: "
DROP    tcp  --  anywhere             anywhere             tcp dpt:http re
cent: CHECK seconds: 30 hit_count: 15 name: httpflood side: source mask: 255.
255.255.255

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

(Insert screenshot of legitimate HTTP request output on Attack VM)

```
┌──(kevin㉿kali)-[~]
└─$ curl http://192.168.10.12:8000
<HTML><BODY>This is your page.</BODY></HTML>
```

(Insert screenshot of HTTP flood output on Attack VM)

```
┌──(kevin㉿kali)-[~]
└─$ sudo hping3 --flood -p 80 -S 192.168.10.12
HPING 192.168.10.12 (eth0 192.168.10.12): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
```

(Insert screenshot of `dmesg` logs for `HTTP flood detected:`)

```
 TTL=64 ID=10308 PROTO=TCP SPT=12573 DPT=80 WINDOW=512 RES=0x00 SYN URGP=0
[ 3997.030714] HTTP flood detected: IN=lo OUT= MAC=00:00:00:00:00:00:00:00:00
:00:00:00:08:00 SRC=192.168.10.12 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00
 TTL=64 ID=43521 PROTO=TCP SPT=12574 DPT=80 WINDOW=512 RES=0x00 SYN URGP=0
[ 3997.030722] HTTP flood detected: IN=lo OUT= MAC=00:00:00:00:00:00:00:00:00
:00:00:00:08:00 SRC=192.168.10.12 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00
 TTL=64 ID=46093 PROTO=TCP SPT=12575 DPT=80 WINDOW=512 RES=0x00 SYN URGP=0
[ 3997.030732] HTTP flood detected: IN=lo OUT= MAC=00:00:00:00:00:00:00:00:00
:00:00:00:08:00 SRC=192.168.10.12 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00
 TTL=64 ID=42208 PROTO=TCP SPT=12576 DPT=80 WINDOW=512 RES=0x00 SYN URGP=0
[ 3997.030740] HTTP flood detected: IN=lo OUT= MAC=00:00:00:00:00:00:00:00:00
:00:00:00:08:00 SRC=192.168.10.12 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00
 TTL=64 ID=41765 PROTO=TCP SPT=12577 DPT=80 WINDOW=512 RES=0x00 SYN URGP=0
[ 3997.030748] HTTP flood detected: IN=lo OUT= MAC=00:00:00:00:00:00:00:00:00
:00:00:00:08:00 SRC=192.168.10.12 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00
 TTL=64 ID=26487 PROTO=TCP SPT=12578 DPT=80 WINDOW=512 RES=0x00 SYN URGP=0
[ 3997.030757] HTTP flood detected: IN=lo OUT= MAC=00:00:00:00:00:00:00:00:00
:00:00:00:08:00 SRC=192.168.10.12 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00
 TTL=64 ID=9027 PROTO=TCP SPT=12579 DPT=80 WINDOW=512 RES=0x00 SYN URGP=0
[ 3997.030765] HTTP flood detected: IN=lo OUT= MAC=00:00:00:00:00:00:00:00:00
:00:00:00:08:00 SRC=192.168.10.12 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00
 TTL=64 ID=14993 PROTO=TCP SPT=12580 DPT=80 WINDOW=512 RES=0x00 SYN URGP=0
[ 3997.030776] HTTP flood detected: IN=lo OUT= MAC=00:00:00:00:00:00:00:00:00
:00:00:00:08:00 SRC=192.168.10.12 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00
 TTL=64 ID=47595 PROTO=TCP SPT=12581 DPT=80 WINDOW=512 RES=0x00 SYN URGP=0

┌──(kevin㉿kali)-[~/Documents/web]
```

Answers to Questions:

52. How many HTTP requests were logged before the Defense VM blocked further requests? *(Provide your answer here)*

The first 15 requests were logged before the rest were dropped.

53. Did the Defense VM successfully mitigate the HTTP flood while allowing normal web traffic? *(Provide your answer here)*

Yes the curl request only generated a. few packets and since it was well within the threshold, the requests were loaded perfectly.

54. What alternative techniques could attackers use to evade this HTTP flood detection? *(Provide your answer here)*

An attacker could use slow attacks and send HTTP headers without triggering the rule.

55. What additional measures can be taken to improve web server protection against HTTP-based DoS attacks? *(Provide your answer here)*

You can use Web Application firewalls to inspect the actual content of the hTTP requests.